

DSA STUDY BLUEPRINT SERIES

108. Detect Cycle in Undirected Graph using BFS

Checking Cycles in Undirected Graphs using BFS

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Store nodes in a queue alongside their parent links.
- Cycle exists if a visited node is reached via a different parent.

Memory & Execution Blueprint

Queue pop element 3 (Parent: 2) • Neighbor 1 is visited • $1 \neq 2 \Rightarrow \text{Cycle!}$

C++ Implementation Reference

Standard code layout and syntax

```
// Queue stores pair (node, parent).  
// Cycle exists if we visit a neighbor that is already visited and is not the parent.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Key Practices & Gotchas

- **Important:** Ensure correct tracking of parent indices to avoid false positives.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.